QuoteVoice

PRODUCT CASE STUDY · FOR RECRUITERS AND HIRING MANAGERS

From a Reddit complaint to a live AI product, end to end.

How I found a real problem, decided what to build (and what not to), designed the system, tested it, secured it and shipped it, with every decision written down.

BUILT BY
Saifullah Khan
saif4rmkhan@gmail.com

LIVE APP
saif-byte.github.io/quotevoice

CODE
github.com/saif-byte/quotevoice

Scan to try it

AT A GLANCE

Painters lose jobs to slow quotes. QuoteVoice turns a voice note into a priced quote.

~3 min
from talking through the job to a quote ready to send (target set before building)

97.4%
reading accuracy on voice notes the reader was never tuned on

$0
a month to run: every service is on a free tier, by design

14
releases shipped, each one tested and deployed automatically

The journey in this document

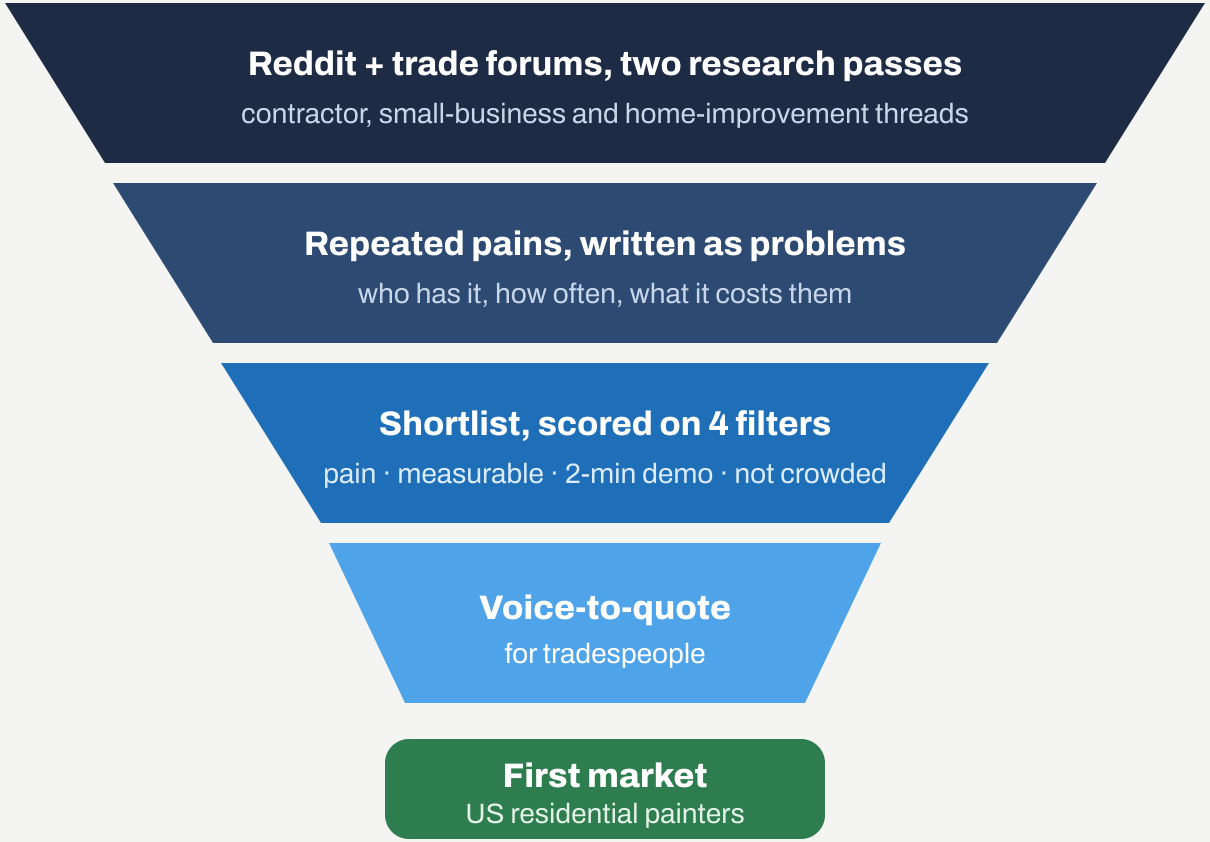
1 **Discovery** real pain
2 **Validation** risky bets
3 **Scope** build / skip
4 **Priorities** risk first
5 **Architecture** stack + why
6 **AI design** AI vs code
7 **Database** secure rules
8 **Quality** measured
9 **New features** 8-step loop
10 **User testing** 15 fixes
11 **Shipping** push = live

PRODUCT THINKING | ENGINEERING | QUALITY + DELIVERY

01 PROBLEM DISCOVERY

Start with complaints, not ideas.

I read Reddit and trade forums for problems small businesses kept repeating, then narrowed them down with four filters before choosing one.



The pain, in one line

Painters walk the job, scribble notes, then **type the quote at night** and send it a day or two later. Homeowners collect several bids, and the **fastest professional-looking quote often wins.**

Why this one (decision D1)

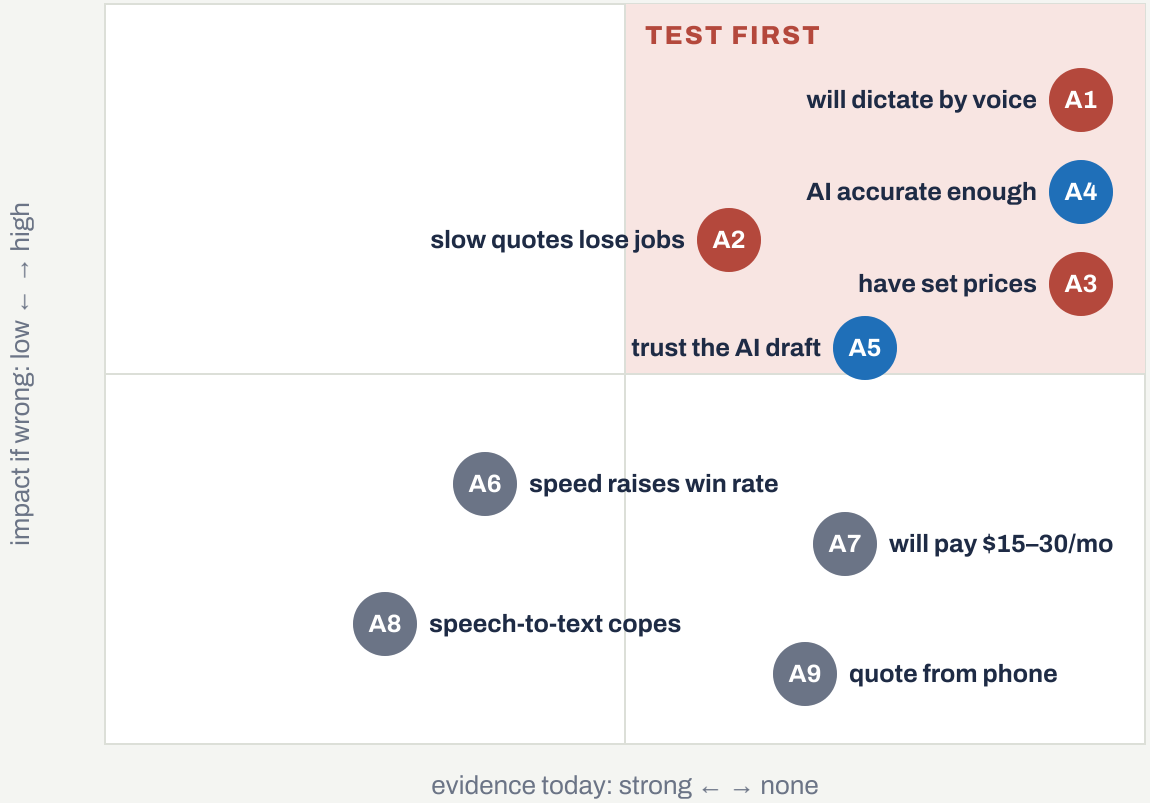
- **Clear, measurable pain:** time to quote and jobs won.
- **Strong 2-minute demo:** talk, and a priced quote appears.
- **Less crowded** than support or follow-up tools I also looked at.

Why painters first? One trade means one price model (sq ft, coats, trim) the system can get right, instead of a weak version for every trade.

02 RESEARCH & VALIDATION

Write down what must be true, then test the riskiest bet first.

Assumption map: 9 bets, ranked by impact and evidence



Experiment E1: problem interviews + a 2-minute voice test

6–8 painters, 20 minutes each. Ask about the **last real quote**, never "would you use...". End by having them describe a job as a voice note, which also becomes test data for the AI.

Pass, set in advance

4 of 6 lost a job to a slow quote, **and** 4 of 6 give a usable voice note in under 2 min, **and** 3 of 6 know their rates.

Fail, set in advance

3+ would never speak a quote aloud, or most price every job from scratch. Then pivot to photos-first or typed notes.

Research changed the design: desk research showed most painters dictate **in the van after the visit**, not in front of the customer, so the app is built for that moment.

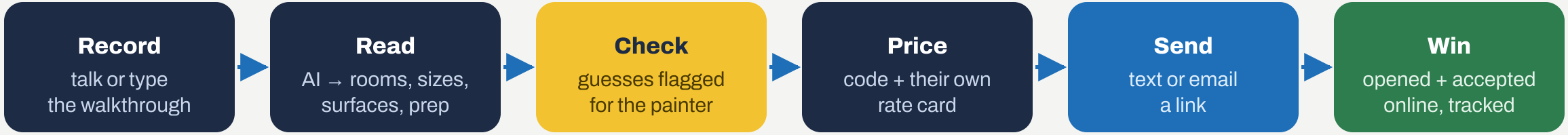
Honest status: the interview plan, script and recruiting posts are ready; the interviews themselves are the next step. Targets were fixed first so results can't be rationalized later.

03 WHAT TO BUILD, AND WHAT NOT TO

One complete loop beats ten half-features.

Rule from the project plan: **every feature must map to an assumption or a metric.** If it doesn't, it waits.

The core loop (built)



Built, and the bet it serves

Voice capture + AI reading	A1 voice works, A4 AI accuracy
Flagged guesses + editable lines	A5 trust: painter stays in control
Rate card in Settings	A3 painters have set rates
Link + online accept + tracking	A6 speed wins jobs, north-star metric
Quality dashboard	edit-free lines, override rate, time to quote

Deliberately not built (yet)

Invoicing, scheduling, CRM	scope creep (risk R5); other tools do it
Photos on site	not needed to test A1/A2; later
Native mobile app	a web app runs on every phone, ships today
Paid SMS / payments	phone's own Messages app is free
AI that sets prices	never: a wrong price is real money

04 PRIORITIZATION

Ship the riskiest learning first, then add what keeps users.

Each release answers one question. The order follows risk: first "does the core loop work at all", then "can it be trusted", then "does it help win the job".



Core loop

Record → read → price → share link. Offline parser as fallback. Dashboard + metrics. **Free stack, no backend.**

Q: can it make a correct quote at all?

Discounts

% or $ off with a reason the customer sees. Kept **out** of AI-quality metrics.

Q: real quotes get negotiated. Without it, painters leave the tool.

Field-test fixes

15 issues from a simulated painter: misreads, an overwrite bug, $0 quotes, blank business details.

Q: would a real painter trust it? Fix before adding.

Accounts + accept

Sign-in, sync across phone and laptop, short links, homeowner accepts online, AI key moved to the server.

Q: can the quote close the job, not just describe it?

Send, track, notify

Text/email in one tap, "opened" tracking, live alert the moment a customer accepts, follow-up list.

Q: does it help the painter follow up and win?

Security review

Attack-tested the database, locked down the AI endpoint, size limits, content policy.

Q: is it safe to put customers' details in?

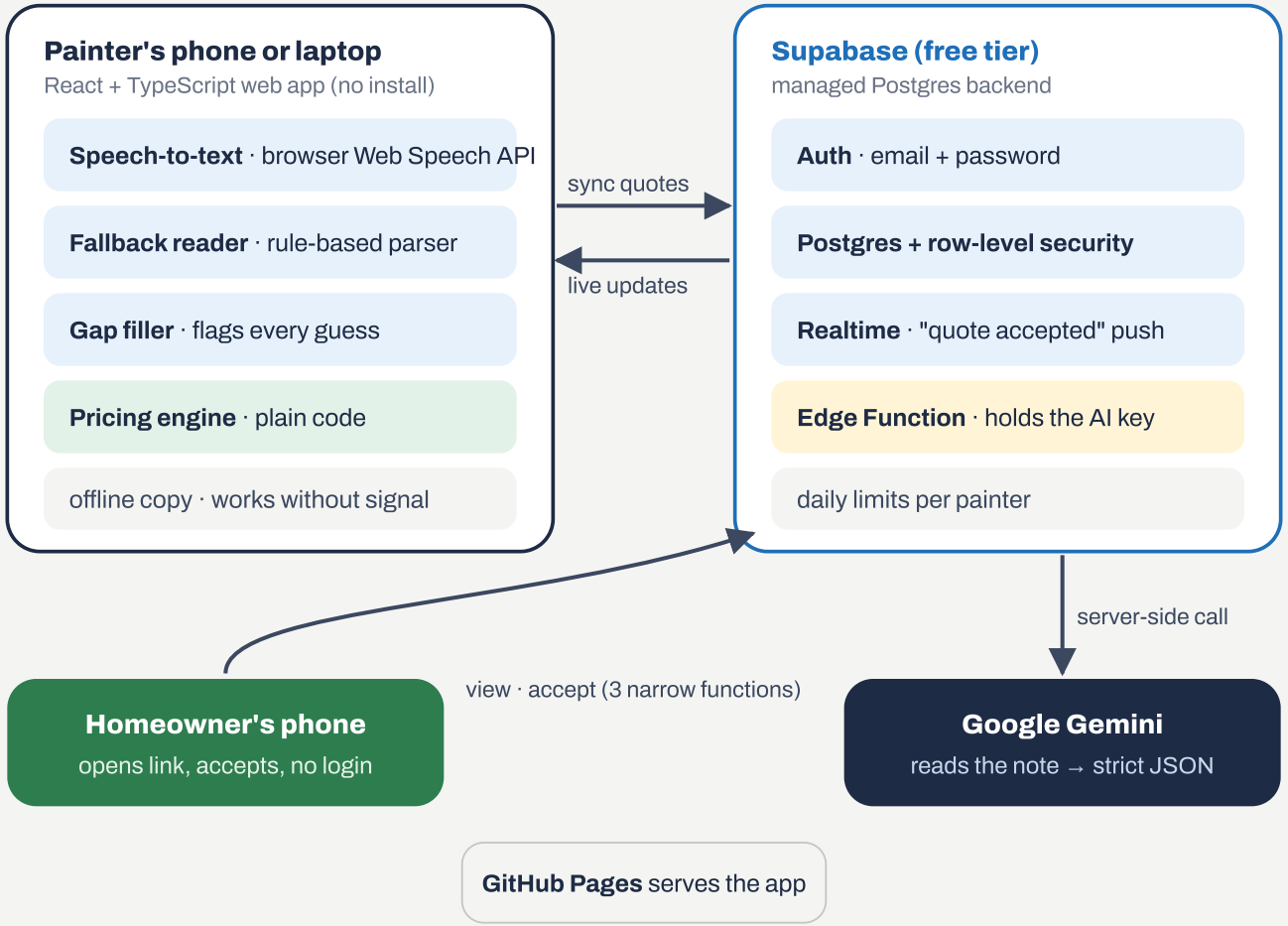
Onboarding + demo

First-sign-in guide that sets up rates for real, a product video, and a self-resetting demo account.

Q: can a new painter get value in the first 2 minutes?

05 TECH STACK & ARCHITECTURE

Simple pieces, each chosen for a reason.



PIECE	WHY THIS
React + TS + Vite	typed data model catches pricing bugs early; static build hosts anywhere
Web Speech API	speech-to-text built into the phone: free, no audio stored
Gemini, JSON schema	free tier; the schema forces valid structured output
Supabase	auth, Postgres, realtime and functions in one free service
GitHub Pages + Actions	every push is tested and deployed, no server to manage

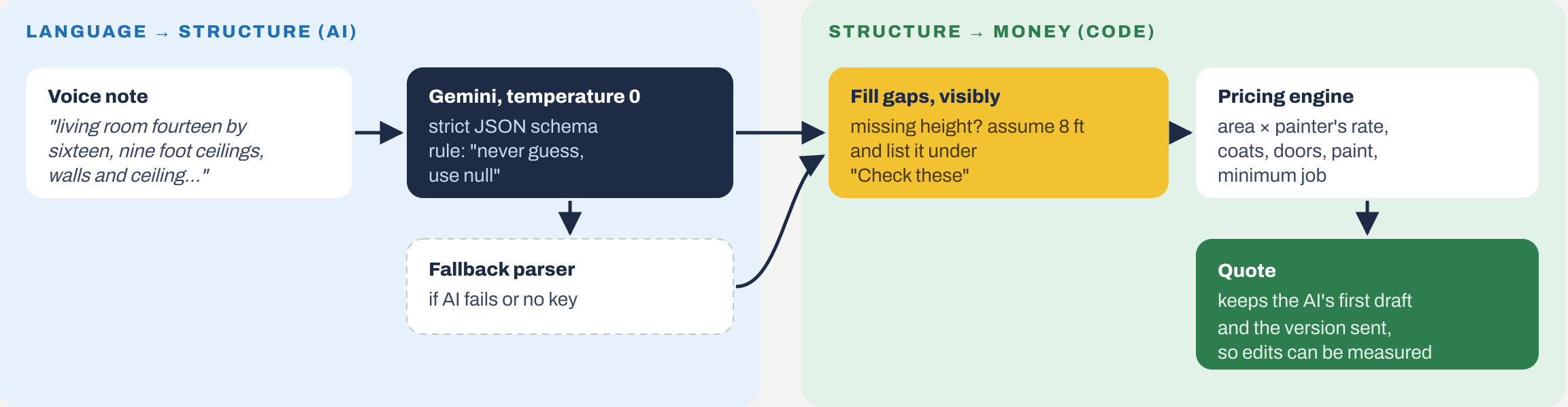
Built to degrade gracefully: no AI key, no signal, or no account? The app still works with the built-in reader and a local copy.

Trade-off accepted: free tiers have limits (AI requests/day, DB pauses when idle). Handled with daily quotas and a keep-alive job.

06 THE KEY DESIGN DECISION

The AI reads the note. Code does the math.

LLMs are great at messy language and unreliable at arithmetic. A wrong price is real money with the painter's name on it, so the pipeline splits the job at that line.



Typed data at every step

`RawJob → JobSpec → LineItem[] → Quote`

"Unknown" is **null**, never a guess, until the gap-filler makes the guess visible.

Swappable model

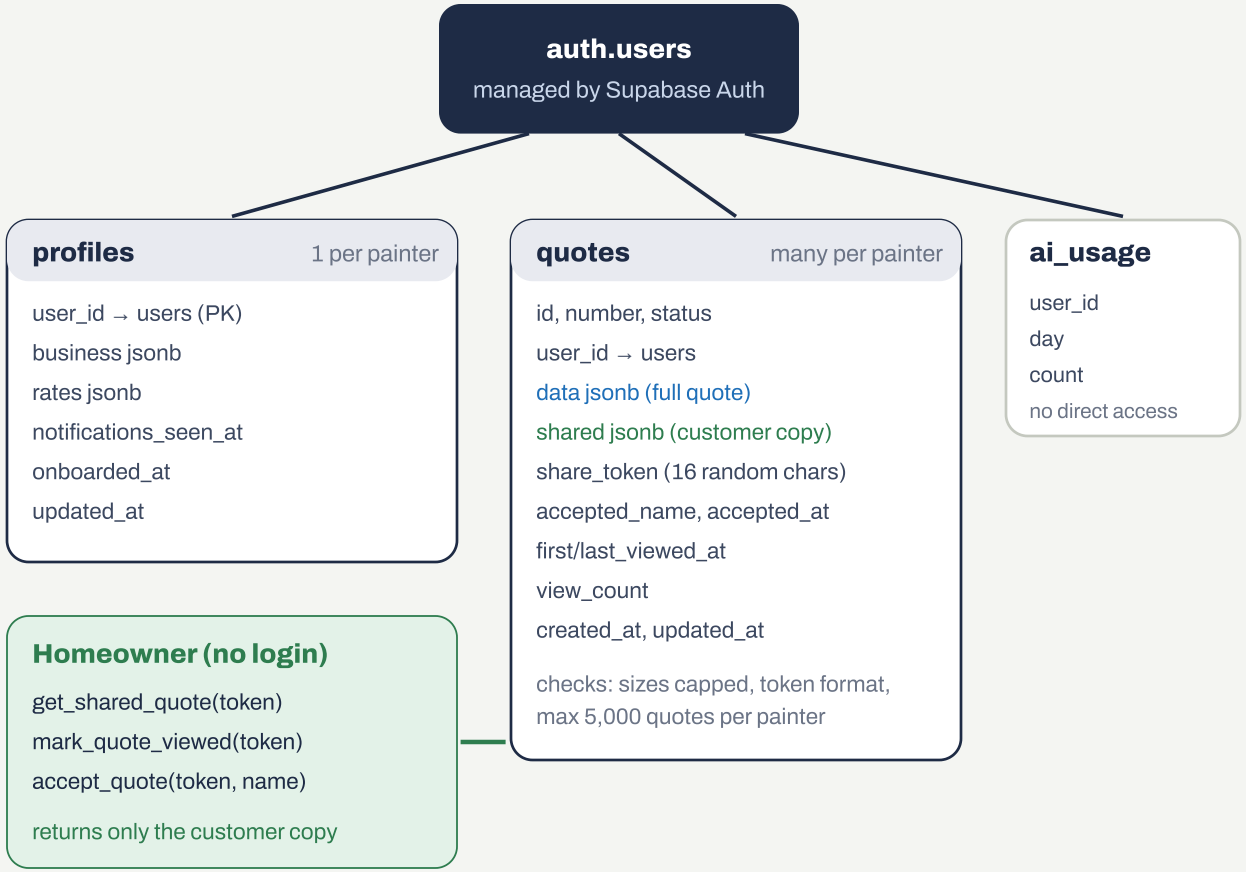
One file of prompt + schema is shared by the browser and the server function, so the model can change without touching pricing (risk R6).

Trust by design

Nothing is ever auto-sent. Every line is editable, every guess is flagged, and the painter's own rates set every price.

07 DATABASE DESIGN

Painters see only their own rows. Homeowners get three narrow doors.



Why JSON for the quote, columns for the rest

The quote's shape changed every release (discounts, flags, tracking). Keeping it as JSON meant **no migrations for product changes**. Only what the database must enforce or search gets a real column: status, link code, acceptance, views.

Two copies, on purpose

data is the painter's full quote, with the AI draft and internal notes. **shared** is the customer copy. A homeowner can only ever reach the shared one.

Rules live in the database, not the app

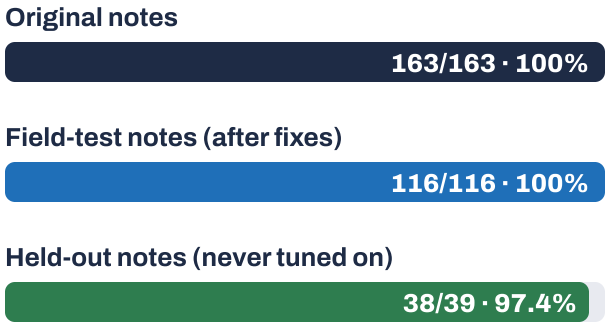
Row-level security on every table. Acceptance can only be set by **accept_quote**, which locks the row and refuses expired estimates, so even a modified app can't fake a "won".

08 QUALITY CHECKS

If AI quality matters, measure it. Three layers of checks.

1 · Offline AI eval

Hand-labeled voice notes, scored field by field (rooms, sizes, surfaces, prep, coats).



The held-out score is the honest one: the first two sets were used to build and fix the reader.

2 · In-product metrics

Every quote stores the AI's first draft and what was sent, so the dashboard can show:

AI lines sent unedited	target ≥ 80%
Totals moved > 10%	target < 20%
Time to quote	target < 3 min
Quotes sent / week	north star

Targets were written before any code, so results can't be rationalized later.

3 · Tests + security

- **33 unit tests** on number parsing, areas, pricing, discounts, metrics and share links
- **Eval runs on every push**: a regression blocks the deploy
- **17 attack checks** on the live database: another painter's data, fake acceptances, oversized data, AI-quota abuse. All blocked.
- **Content Security Policy**, crafted-link cleaning, weekly dependency alerts

Before/after on real phrasing: the field test found notes the reader got wrong (spoken corrections, "we did their kitchen last year", "ceilings are like fourteen foot"). Accuracy on those notes went from **73.3% → 100%**, then a fresh held-out set confirmed it generalizes (97.4%).

SECURITY

Secured in the database, then attacked to prove it.

Rules live in the database, where a modified app or a direct API call can't skip them. I attacked them as an outsider and as a second painter: **17 of 17 checks blocked.**

ATTACK	PROTECTION	CHECKED BY
Reading another painter's quotes	Row-level security on every table; anonymous visitors get no table access at all	signed in as painter B, tried read / edit / delete / insert as A
Faking "quote accepted"	Only `accept_quote` can accept; it locks the row and refuses expired quotes; a trigger blocks painters editing acceptance or views	tried it as the painter and on an expired quote
Guessing customer links	16 random characters (about 92 bits), format-checked; only the customer copy is returned	malformed and wrong codes
Stealing or draining the AI key	Key only on the server; sign-in required; 100 notes per painter, 1,500 per day in total	anonymous call, 101st request
Injected scripts, crafted links	React escaping, Content Security Policy, link data cleaned to expected fields; mail and phone links only from valid values	a crafted link with script and fake email
Clickjacking	The app refuses to run inside another site's frame	framed the app in a test page
Flooding the free database	Size limits on every field, 5,000 quotes per painter, 4,000-character notes	oversized quote insert
Hijacking the shared demo login	Email and password can't be changed; data resets when idle	password-change attempt

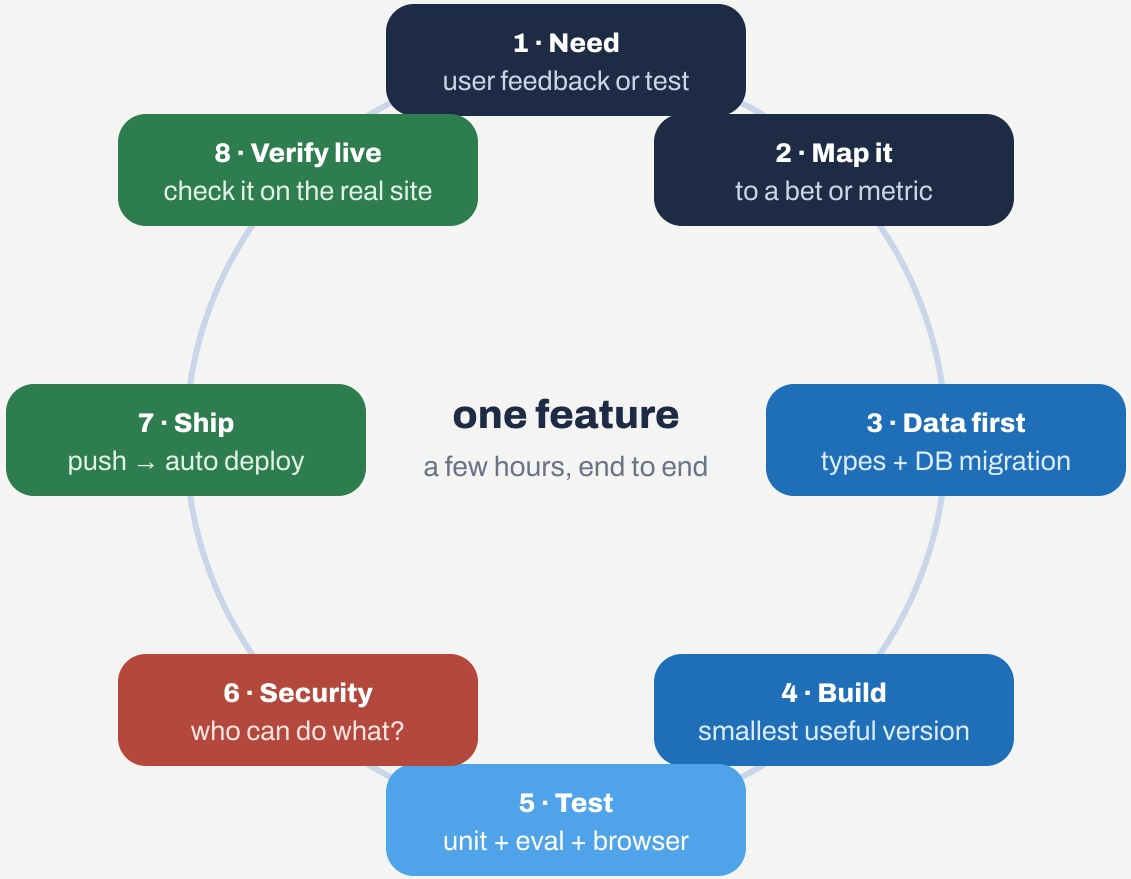
Kept secure over time

- **0 known vulnerabilities** in packages at review
- **Dependabot** checks packages and CI weekly
- **No secrets in code** or git history; only the public browser key ships
- Every new feature passes a **"who can do what?"** step before shipping

Owner settings outside the code (password rules, CAPTCHA, AI key on free tier) are listed in SECURITY.md in the repo.

09 HOW NEW FEATURES ARE ADDED

The same eight steps for every feature.



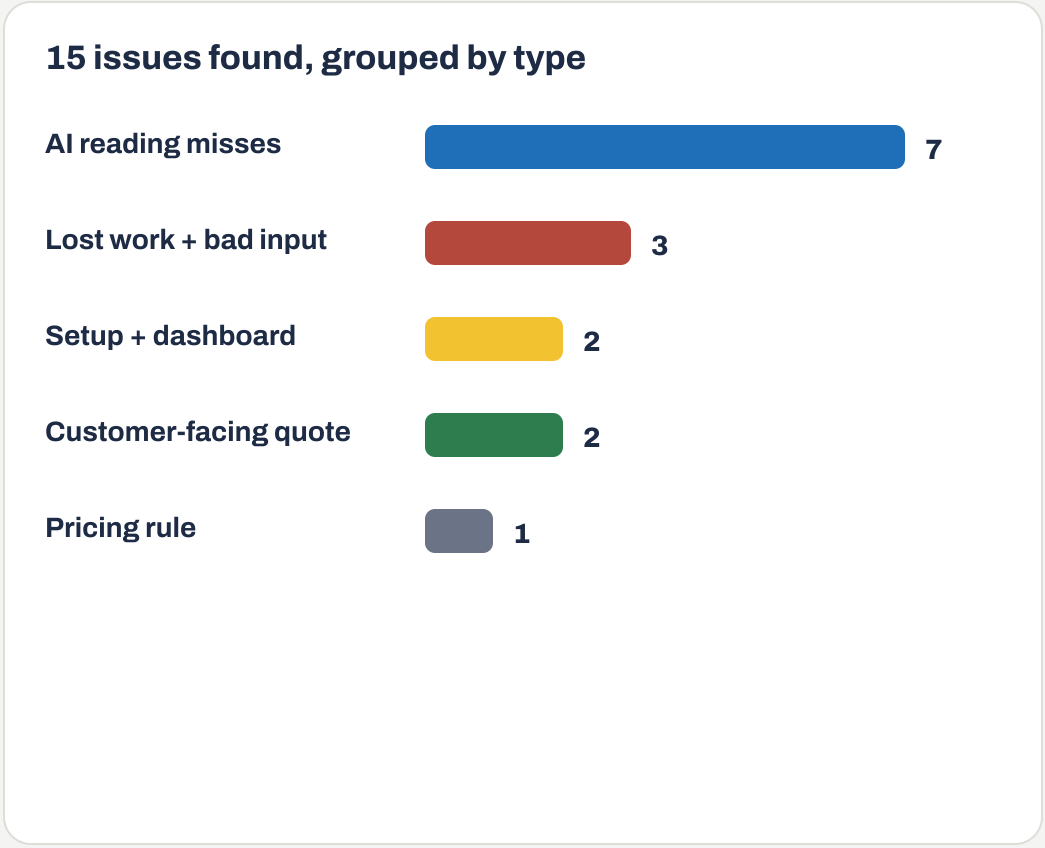
Worked example: "tell me when a customer accepts"

1 · Need	"If someone accepts my quote I should get a notification on my dashboard."
2 · Map it	Quotes won, and fast follow-up (A6: speed wins jobs).
3 · Data first	One new column, `notifications_seen_at`, so read/unread syncs across devices; realtime turned on for quotes.
4 · Build	Bell with unread count, live pop-up, "accepted" card, optional device alerts, once-a-minute check if live updates are blocked.
5 · Test	Simulated a homeowner accepting while the painter's app was open; checked the alert, count and title.
6 · Security	Painters can't fake their own acceptance; a database trigger protects those fields.
7–8 · Ship	Pushed, tests passed, deployed, then confirmed on the live site.

10 USER TESTING

A full field test before adding anything new.

Before real painters, an AI agent used the live app as a persona ("Mike Delgado, residential painter") with real-world tasks, and reported everything that went wrong. Every issue was fixed in v1.1.



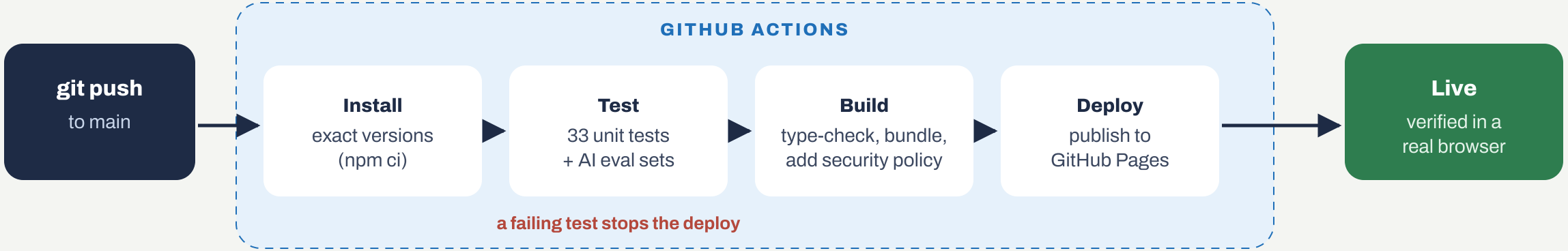
FOUND	FIXED
"Fourteen by twelve no wait twelve by twelve" read as 14×12	spoken corrections handled
"We did their kitchen last year" priced the kitchen	past and turned-down work skipped
Editing a quote, then New quote, overwrote it	separate routes per quote
$0 quotes and negative numbers allowed	blocked
New accounts showed an example company's name	business details start blank
Dashboard led with AI stats, not money	leads with open value, won, follow-ups

What came out of it beyond fixes: a first-sign-in guide (business details and prices set up in 2 minutes) and a "Check these" list for every guess.

Next: the 6–8 real painter interviews (E1). Their voice notes go into the eval set, replacing written examples.

11 DEPLOYMENT PIPELINE

Push to main → tested and live in minutes.



Database changes

Versioned SQL migration files in the repo, each one applied and then attack-tested.

Server function

The AI function's code lives in the repo; a test checks it matches the browser's copy.

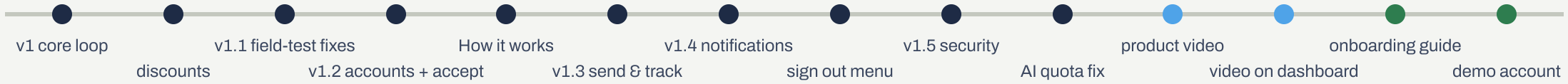
Scheduled jobs

Daily keep-alive so the free database never pauses; the demo account resets itself when idle.

Maintenance

Dependabot opens weekly update PRs; secrets live only in GitHub and Supabase, never in code.

Release history: 14 releases, every one through this pipeline



TRY IT YOURSELF

Two minutes in the demo account.

Demo login (resets itself after 30 idle minutes)

Website	**saif-byte.github.io/quotevoice**
Email	`demo@quotevoice.test`
Password	`QuoteVoice-demo-2026`

Or press **Try the demo account** in the recruiter section of the site.

> **Let's talk:** Saifullah Khan · **saif4rmkhan@gmail.com**

Suggested tour

1. **Dashboard:** money waiting, wins, the bell with a fresh "accepted" alert.
2. **New quote → a sample note → Build quote:** watch the note become rooms, with guesses flagged.
3. **Price this job:** every line from the rate card; add a discount.
4. **Open Tina Ruiz's quote → copy the customer link** in a private window and accept it as the homeowner. The painter's app updates live.

What I learned

- **Decide the metric before the feature.** "AI lines sent unedited" shaped the data model on day one.
- **Put the AI where it's strong.** Language in, structure out; arithmetic stays in code.
- **Hold-out data keeps you honest.** 100% on tuned data meant little; 97.4% on unseen notes meant something.
- **Security belongs in the database.** Rules the app can't bypass, tested by attacking them.
- **Free is a design constraint, not an excuse.** Quotas, keep-alive and fallbacks made it production-shaped.

What's next

- Run the painter interviews and feed real voice notes into the eval.
- Score Gemini against the built-in reader on the same sets.
- Photos on site, then phone push notifications.